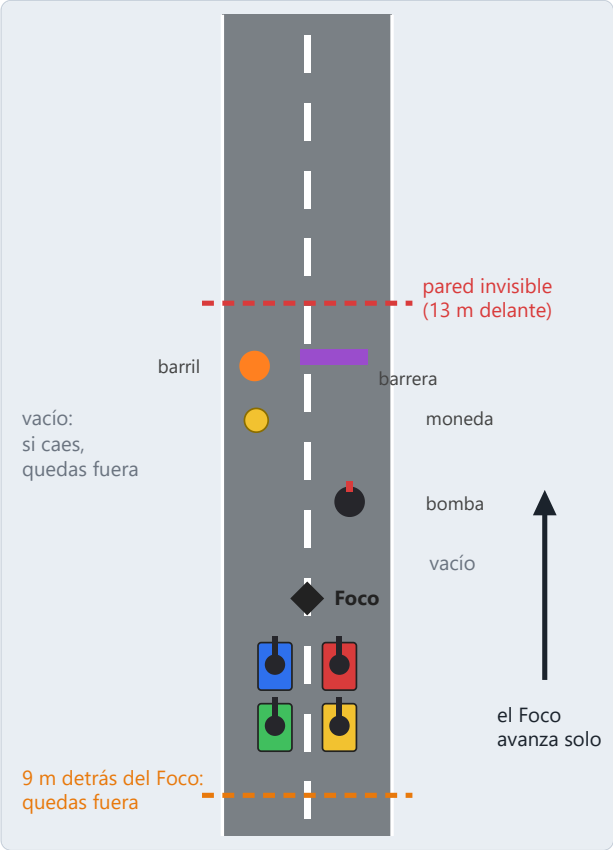
PROGRAMACIÓN PARA VIDEOJUEGOS · UNITY

Subida Chocona

Guía del profesor: qué explicar, qué preguntar y qué esperar en cada paso, desde el paquete de Inicio hasta el juego terminado.

El juego en una frase

Hasta 4 tanques van apretados por una carretera de montaña que "sube" sola. Se empujan, recogen monedas, esquivan barriles y barreras y huyen de las bombas. Quien se cae o se queda atrás queda fuera. Gana el último que siga.



Vista desde arriba. La carretera está quieta: lo que avanza es el Foco, y con él la cámara.

¿Cómo se mueve "el mapa"?

No se mueve el mapa: se mueve la cámara. Visto en la pantalla se ve igual que si el mapa bajara hacia los tanques, pero la física se queda estable. Los tanques pueden ir más rápido que la cámara; si se quedan atrás, quedan fuera.

Qué hay en el paquete

Qué	Contenido
Prefabs	Tanque, Carretera (tramo de 40 m), Moneda, Bomba, Barril, Barrera
Scripts vacíos	TanqueRuta, CamaraRuta, MarcadorRuta (solo traen la meta escrita)
Escena **Ruta_Vacia**	Solo el Foco con la cámara, la luz y el marcador en pantalla. Aquí arman su entorno.
Escena **Ruta_Inicio**	El nivel ya armado, con 86 objetos. Es el plan B para quien no termine el paso 0.

Importar: Assets, Import Package, Custom Package. Requisitos: Unity 6 con plantilla URP y Active Input Handling en "Both". El paquete crea solo los tags Moneda y Bomba.

Los tres scripts

Son tres. Ojo: **Foco** no es un script, es un objeto vacío. El script que lo mueve se llama **CamaraRuta**.

Script	Va en	Qué hace
TanqueRuta	cada Tanque (4)	Mueve el tanque, recoge monedas, explota bombas y lo saca de la partida.
CamaraRuta	el objeto Foco (1)	Hace avanzar la cámara por la carretera.
MarcadorRuta	el objeto Marcador (1)	Escribe los puntos y anuncia al ganador.

Plan de tiempos

Paso	Qué	Minutos
0	Armar el entorno	30
1	Que un tanque avance y retroceda	15
2	Que el tanque gire	10
3	Cuatro jugadores con el mismo script	15
4	Que la cámara se mueva	25
5	Monedas	15
6	Bombas	25
7	Quedar fuera: caerse o quedarse atrás	20
8	Marcador y ganador	25
9	Jugar, equilibrar y mejorar	30
	Total	**210**

Es mucho para una sola clase. Sugerencia: **clase A** pasos 0 a 4 (95 min) y **clase B** pasos 5 a 9 (115 min). Al terminar la clase A ya se puede jugar a correr y empujarse.

Qué practican

Paso	Conceptos
0	Prefabs, Inspector, collider sólido y trigger.
1	Input.GetKey, KeyCode, Rigidbody, Update y FixedUpdate.
2	Rotación, Quaternion.Euler, Time.fixedDeltaTime.
3	Reutilizar un script, variables públicas, colisiones entre Rigidbody.
4	Time.deltaTime, variables que cambian con el tiempo, condiciones.
5	OnTriggerEnter, CompareTag, Destroy.
6	OnTriggerEnter, Physics.OverlapSphere, ciclo for, AddExplosionForce.
7	Comparar posiciones, bool, SetActive.
8	Referencias entre objetos, TextMeshPro, métodos con parámetros, contar.
9	Probar, ajustar valores, ideas propias.

0 Armar el entorno

30 minutos · Prefabs, Inspector, collider sólido y trigger.

LO QUE DIGO

- Les paso un paquete con las piezas. Ustedes arman la carretera, ponen los 4 tanques y reparten los obstáculos. Tienen media hora.
- Todavía no hay código: ahora solo estamos armando el escenario. Pero miren cada pieza en el Inspector, porque después cada una se va a comportar distinto.
- La carretera es una sola tira larga. Se arma duplicando un tramo de 40 metros.

PREGUNTAS PARA EL GRUPO

- ¿Cuáles piezas tienen Is Trigger activado y cuáles no?
- ¿Cuáles tienen Rigidbody?
- ¿Qué tag tiene cada una?

LO QUE HACEN ELLOS

1. Importar el paquete: Assets, Import Package, Custom Package. Abrir la escena **Ruta_Vacia**.
2. Arrastrar el prefab **Carretera** a la escena y dejarlo en Position (0, 0, 0). Duplicarlo con Ctrl+D y subir cada copia 40 en Z: 40, 80, 120 y 160. Quedan 5 tramos, 200 metros.
3. Arrastrar 4 prefabs **Tanque** y nombrarlos Tanque_J1 a Tanque_J4. Posiciones: J1 (-1.5, 0.45, -1), J2 (1.5, 0.45, -1), J3 (-1.5, 0.45, -3.5), J4 (1.5, 0.45, -3.5).
4. Dar un color a cada tanque: dentro de cada Tanque, el hijo **Cuerpo**, y arrastrar el material Azul (J1), Rojo (J2), Verde (J3) o Amarillo (J4).
5. Repartir por la carretera, entre Z = 20 y Z = 190: 10 **Moneda**, 6 **Bomba**, 8 **Barril** y 6 **Barrera**. Con X entre -3 y 3. Alturas Y: Moneda 0.8, Bomba 0.45, Barril 0.6, Barrera 0.5.
6. Darle Play. No debe pasar nada: los tanques quedan quietos.

Pistas (sin dar el código)

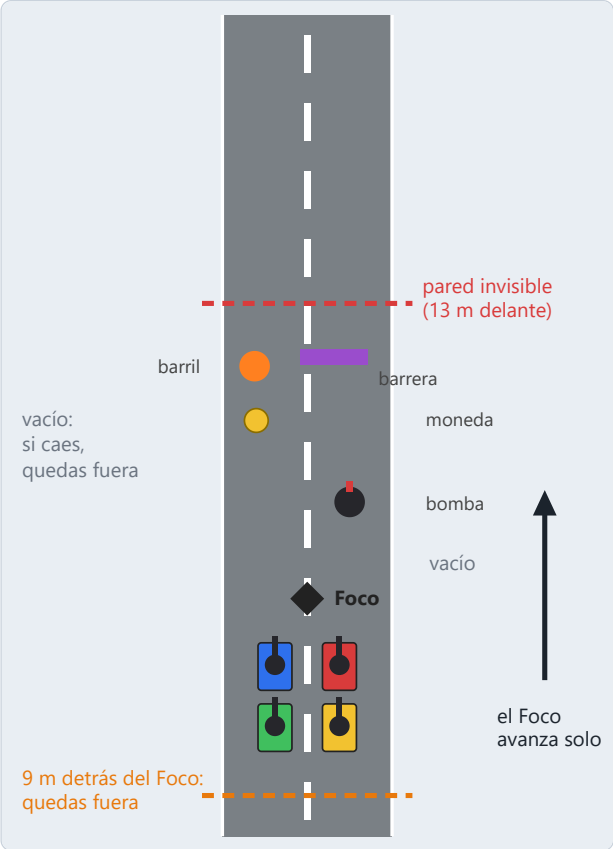
- Ctrl+D duplica el objeto seleccionado.
- Si un tanque queda hundido o flotando, revisar su Y.
- Para ver las propiedades de una pieza: seleccionarla y mirar el Inspector.

Cómo compruebo que funciona

- En Play no se mueve nada y la cámara ve la carretera en diagonal.
- Todos los tanques están sobre la carretera, apretados en la parte de abajo.
- Cada pieza está sobre el asfalto, no enterrada ni flotando.

ERRORES TÍPICOS

- Tanque o ítems con la Y mal puesta: quedan enterrados bajo la carretera o en el aire.
- Mover el Foco o la Main Camera sin querer.
- Quien no alcance: abrir la escena **Ruta_Inicio**, que ya trae la carretera y los 86 objetos colocados. Es el plan B.



Así debe quedar, visto desde arriba: tanques abajo, objetos repartidos y el Foco en el medio.

Pieza	Collider	Tipo	Rigidbody	Tag
Tanque	caja	sólido	sí (rotación X y Z congelada)	-
Carretera	caja	sólido	no	-
Barril	cápsula	sólido	no	-
Barrera	caja	sólido	no	-
Moneda	esfera	trigger	no	Moneda
Bomba	esfera	trigger	no	Bomba

1 Que un tanque avance y retroceda

15 minutos · Input.GetKey, KeyCode, Rigidbody, Update y FixedUpdate.

LO QUE DIGO

- Cada tanque se controla con su propio teclado. Por eso la tecla no puede estar escrita fija en el código: tiene que ser una variable que se elige en el Inspector.
- Hay varias formas de mover algo con Rigidbody: MovePosition, AddForce o cambiar la velocidad. Escojan la que quieran. Yo uso AddForce porque así los choques se sienten naturales.
- Regla: las teclas se leen en Update y la fuerza se aplica en FixedUpdate.

PREGUNTAS PARA EL GRUPO

- ¿Qué tipo de variable guarda una tecla?
- ¿Hacia dónde apunta el tanque?
- ¿Qué pasa cuando suelto la tecla?

LO QUE HACEN ELLOS

1. Abrir **TanqueRuta.cs**.
2. Agregar variables públicas para la tecla de adelante, la de atrás y la fuerza.
3. Leer las teclas en Update y guardar si el tanque quiere ir adelante (1), atrás (-1) o quedarse quieto (0).
4. Aplicar la fuerza en FixedUpdate.
5. Poner el script solo en Tanque_J1 y elegir sus teclas en el Inspector (W y S).

Pistas (sin dar el código)

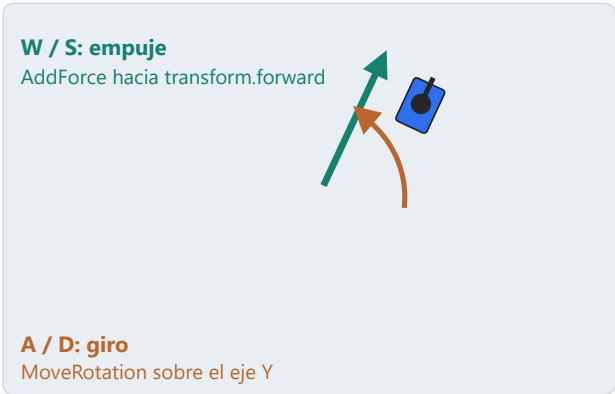
- Una tecla se guarda en una variable de tipo KeyCode.
- Para mover un Rigidbody hace falta guardarlo en Start con GetComponent.
- La dirección en la que apunta el tanque es transform.forward.
- ForceMode.Acceleration hace que la masa no importe.

Cómo compruebo que funciona

- En Play, W hace avanzar el tanque azul y S lo hace retroceder.
- Al soltar la tecla el tanque se frena solo (lo hace el Linear Damping del Rigidbody).

ERRORES TÍPICOS

- Las teclas quedan en None en el Inspector: el tanque no responde.
- NullReference al darle Play: olvidaron guardar el Rigidbody en Start.
- Mover con transform.position: el tanque atraviesa todo.



El empuje va hacia donde apunta el cañón. Lo que cambia al girar es hacia dónde apunta.

▼ Solución del paso 1 (mostrar al final)

TanqueRuta.cs: variables nuevas

```
    public KeyCode teclaAdelante;
    public KeyCode teclaAtras;
    public float fuerza = 28f;

    private Rigidbody rb;
    private float empuje = 0f;
```

TanqueRuta.cs: Start, Update y FixedUpdate

```
    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        empuje = 0f;
        if (Input.GetKey(teclaAdelante))
        {
            empuje = 1f;
        }
        if (Input.GetKey(teclaAtras))
        {
            empuje = -1f;
        }
    }

    void FixedUpdate()
    {
        rb.AddForce(transform.forward * empuje * fuerza, ForceMode.Acceleration);
    }
```

2 Que el tanque gire

10 minutos · Rotación, Quaternion.Euler, Time.fixedDeltaTime.

LO QUE DIGO

- Girar es cambiar hacia dónde apunta el tanque. Como el empuje siempre va hacia transform.forward, si giro el tanque, el empuje también cambia de dirección.
- No hay que entender los Quaternion a fondo. Quaternion.Euler(0, ángulo, 0) significa "gira ese ángulo alrededor del eje Y".

PREGUNTAS PARA EL GRUPO

- ¿Qué pasaría si usan Vector3.forward en vez de transform.forward?
- ¿Por qué el giro se multiplica por el tiempo?

LO QUE HACEN ELLOS

1. Agregar las teclas de izquierda y derecha y una velocidad de giro.
2. Leer esas teclas en Update y guardar si gira a la derecha (1), a la izquierda (-1) o no gira (0).
3. En FixedUpdate, rotar el Rigidbody ese ángulo.

Pistas (sin dar el código)

- Rigidbody.MoveRotation recibe la rotación nueva.
- La rotación nueva es la actual multiplicada por una rotación pequeña de este instante.
- Time.fixedDeltaTime es el tiempo entre dos pasos de la física.

Cómo compruebo que funciona

- A y D giran el tanque en su lugar.
- Al girar y apretar W, el tanque avanza hacia donde apunta.

ERRORES TÍPICOS

- El tanque se voltea o rueda: se les quitó el Freeze Rotation en X y Z del Rigidbody.
- Gira rapidísimo: olvidaron multiplicar por el tiempo.

▼ Solución del paso 2 (mostrar al final)

TanqueRuta.cs: variables nuevas

```
    public KeyCode teclaIzquierda;
    public KeyCode teclaDerecha;
    public float velocidadGiro = 120f;

    private float giro = 0f;
```

TanqueRuta.cs: en Update, debajo de la lectura de adelante y atrás

```
        giro = 0f;
        if (Input.GetKey(teclaDerecha))
        {
            giro = 1f;
        }
        if (Input.GetKey(teclaIzquierda))
        {
            giro = -1f;
        }
```

TanqueRuta.cs: en FixedUpdate, debajo de AddForce

```
        Quaternion rotacion = Quaternion.Euler(0f, giro * velocidadGiro * Time.fixedDeltaTime, 0f);
        rb.MoveRotation(rb.rotation * rotacion);
```

3 Cuatro jugadores con el mismo script

15 minutos · Reutilizar un script, variables públicas, colisiones entre Rigidbody.

LO QUE DIGO

- Un solo script, cuatro tanques. Lo que cambia entre un tanque y otro no es el código: es la información, o sea las teclas.
- Ahora que juegan varios, choquen entre ustedes. Fíjense quién empuja a quién.

PREGUNTAS PARA EL GRUPO

- ¿Hay que escribir 4 scripts?
- ¿Qué cambia si un tanque pesa 3 en vez de 1 (Mass)?
- ¿Y si subo o bajo el Linear Damping?

LO QUE HACEN ELLOS

1. Poner el script TanqueRuta en Tanque_J2, J3 y J4.
2. Elegir las teclas de cada uno (ver el dibujo del teclado).
3. Jugar entre todos en el mismo teclado y chocarse.
4. Probar cambiar la Mass y el Linear Damping de un tanque y ver qué pasa.

Pistas (sin dar el código)

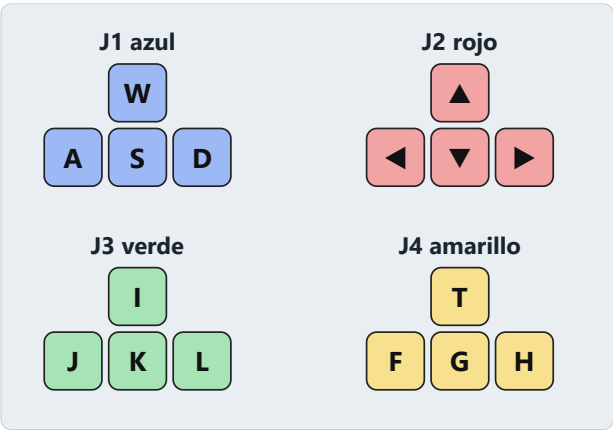
- Se puede copiar el componente de un tanque a otro, o arrastrar el script al Inspector del otro.
- No repetir teclas entre jugadores.

Cómo compruebo que funciona

- Los 4 tanques se mueven al mismo tiempo, cada uno con sus teclas.
- Cuando chocan, se empujan.

ERRORES TÍPICOS

- Dos tanques con las mismas teclas: se mueven juntos.
- Teclas en None.



Las teclas de los cuatro jugadores. J1 y J2 usan las de siempre; J3 y J4 usan IJKL y TFGH.

4 Que la cámara se mueva

25 minutos · Time.deltaTime, variables que cambian con el tiempo, condiciones.

LO QUE DIGO

- La carretera está quieta. Lo que se mueve es la cámara. Visto en la pantalla es igual a que el mapa baje hacia nosotros.
- Foco no es un script: es un objeto vacío que lleva la cámara y una pared invisible. El script que lo mueve se llama CamaraRuta y se pone en el Foco.
- Quiero que arranque despacio, que acelere poco a poco y que se detenga cuando llegue al final de la carretera.

PREGUNTAS PARA EL GRUPO

- ¿Qué pasa si no multiplico por Time.deltaTime?
- ¿Por qué la pared invisible es hija del Foco?
- ¿Cómo hago que espere unos segundos antes de empezar?

LO QUE HACEN ELLOS

1. Abrir **CamaraRuta.cs**. En la escena Ruta_Vacia ya está puesto en el objeto **Foco**. Debe estar en el Foco y no en la cámara.
2. Hacer que el Foco avance en Z con una velocidad, después de esperar unos segundos.
3. Hacer que la velocidad aumente poco a poco hasta un máximo.
4. Hacer que se detenga al llegar a una Z final. Con 5 tramos de carretera, usar 180.
5. Jugar: ahora hay que mantener W apretada para seguir el ritmo.

Pistas (sin dar el código)

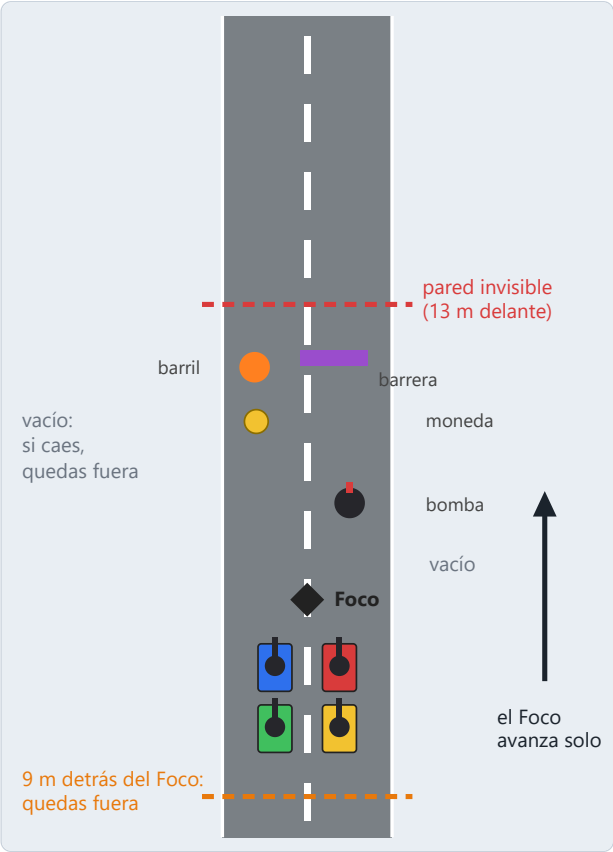
- Vector3.forward es el eje Z.
- Una variable que cuenta el tiempo ayuda a esperar: se le suma Time.deltaTime en cada Update.
- Para aumentar la velocidad: velocidad += aceleracion * Time.deltaTime.
- La Z final debe ser menor que el largo de la carretera.

Cómo compruebo que funciona

- Después de unos segundos la cámara empieza a avanzar sola y cada vez más rápido.
- Los tanques se quedan atrás si sueltan W.
- Quien intenta adelantarse choca con la pared invisible.
- Al llegar a la Z final todo se detiene.

ERRORES TÍPICOS

- El script está en la Main Camera y no en el Foco: se mueve la cámara pero no la pared invisible.
- La velocidad es enorme: olvidaron multiplicar por Time.deltaTime.
- La Z final es menor que la Z actual: el Foco nunca arranca.



El Foco avanza por la carretera. La cámara y la pared invisible se mueven con él.

```
Foco                 (objeto vacío, lleva el script CamaraRuta)
  Main Camera        (hija del Foco, mira la carretera en diagonal)
  ParedInvisible     (hija del Foco, frena a quien se adelante)
Carretera            (5 tramos de 40 m, del 0 al 160 en Z)
Tanque_J1 ... J4     (cada uno con TanqueRuta)
Moneda, Bomba, Barril, Barrera   (repartidos por la carretera)
Canvas
  TextoJ1 ... TextoJ4, TextoCentral
Marcador             (objeto vacío, lleva el script MarcadorRuta)
```

▼ Solución del paso 4 (mostrar al final)

CamaraRuta.cs completo

```
using UnityEngine;

public class CamaraRuta : MonoBehaviour
{
    public float velocidadInicial = 4f;
    public float aceleracion = 0.03f;
    public float velocidadMaxima = 9f;
    public float retardoInicio = 3f;
    public float zFinal = 380f;

    private float velocidad;
    private float tiempo = 0f;

    void Start()
    {
        velocidad = velocidadInicial;
    }

    void Update()
    {
        tiempo += Time.deltaTime;

        if (tiempo > retardoInicio && transform.position.z < zFinal)
        {
            if (velocidad < velocidadMaxima)
            {
                velocidad += aceleracion * Time.deltaTime;
            }

            transform.position += Vector3.forward * velocidad * Time.deltaTime;
        }
    }
}
```

5 Monedas

15 minutos · OnTriggerEnter, CompareTag, Destroy.

LO QUE DIGO

- Un objeto sólido frena y empuja. Un trigger no frena: atraviesa y avisa. La moneda es un trigger.
- El aviso le llega al tanque, porque el tanque tiene Rigidbody. Por eso el script que recoge la moneda va en el tanque, no en la moneda.
- Cómo sabe el tanque que lo que tocó es una moneda: por su tag.

PREGUNTAS PARA EL GRUPO

- ¿Por qué la moneda no frena al tanque?
- ¿Qué tiene que tener el tanque para que le llegue el aviso?
- ¿Para qué sirve el tag?

LO QUE HACEN ELLOS

1. En TanqueRuta, escribir la función que se ejecuta cuando el tanque entra en un trigger.
2. Si lo que tocó tiene el tag Moneda: sumar 1 a puntos y destruir la moneda.
3. Jugar y pasar por encima de monedas.

Pistas (sin dar el código)

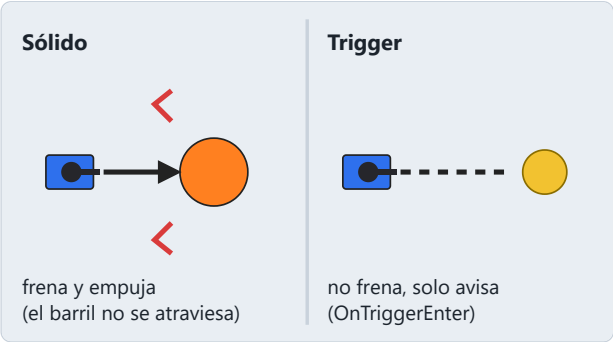
- La función se llama OnTriggerEnter y recibe un Collider.
- Para comparar tags: other.CompareTag("Moneda").
- Destroy(other.gameObject) borra la moneda.

Cómo compruebo que funciona

- La moneda desaparece al pasar por encima.
- Durante Play, seleccionar un tanque en el Hierarchy: la variable Puntos del Inspector sube.

ERRORES TÍPICOS

- El tag mal escrito (moneda con minúscula): nunca la detecta.
- Pusieron el script en la moneda en vez del tanque.
- Destroy(other) en vez de Destroy(other.gameObject): solo borra el componente.



Sólido frente a trigger. Es la idea central de este paso y del siguiente.

▼ Solución del paso 5 (mostrar al final)

TanqueRuta.cs: función nueva

```
    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Moneda"))
        {
            puntos++;
            Destroy(other.gameObject);
        }
    }
```

6 Bombas

25 minutos · OnTriggerEnter, Physics.OverlapSphere, ciclo for, AddExplosionForce.

LO QUE DIGO

- Una bomba es como una moneda: es un trigger con su propio tag. Pero en vez de dar puntos, empuja.
- Para empujar a todos los que están cerca, primero buscamos todos los colliders dentro de una esfera y después empujamos el Rigidbody de cada uno.
- Unity ya trae una función para empujar desde un punto: AddExplosionForce.

PREGUNTAS PARA EL GRUPO

- ¿Qué pasa con los colliders que no tienen Rigidbody, como la carretera o los barriles?
- ¿Qué pasa si no destruyo la bomba?
- ¿Quién sale más empujado: el que está pegado a la bomba o el que está lejos?

LO QUE HACEN ELLOS

1. Agregar variables para el radio y la fuerza de la explosión.
2. En OnTriggerEnter, si lo que tocó tiene el tag Bomba, llamar a una función Explotar.
3. En Explotar: buscar todos los colliders cerca de la bomba, recorrerlos y empujar los que tengan Rigidbody. Al final, destruir la bomba.
4. Jugar y chocar contra una bomba con un tanque cerca de otro.

Pistas (sin dar el código)

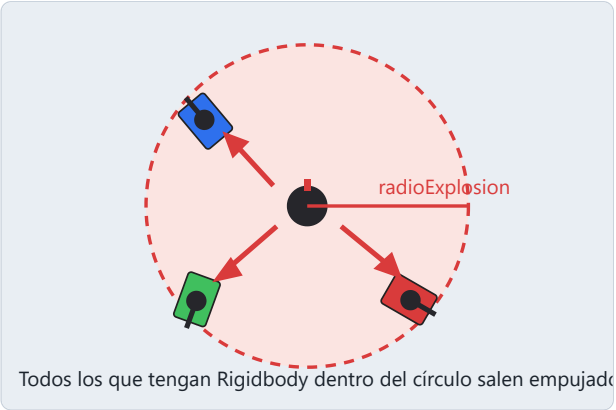
- Physics.OverlapSphere(centro, radio) devuelve un arreglo de Collider.
- collider.attachedRigidbody es el Rigidbody de ese collider, o null si no tiene.
- rigidbody.AddExplosionForce(fuerza, centro, radio, 0.3f, ForceMode.Impulse).
- Recorrer el arreglo con un ciclo for.

Cómo compruebo que funciona

- Al tocar una bomba el tanque sale empujado y la bomba desaparece.
- Los tanques cercanos también salen empujados.
- Un tanque lejos de la bomba no se mueve.

ERRORES TÍPICOS

- Radio en 0: no empuja a nadie.
- No destruyeron la bomba: explota muchas veces seguidas.
- No revisaron si el Rigidbody es null: error al llegar a un collider sin Rigidbody.



La explosión empuja desde el centro hacia afuera, solo dentro del radio.

▼ Solución del paso 6 (mostrar al final)

TanqueRuta.cs: variables nuevas

```
    public float radioExplosion = 4f;
    public float fuerzaExplosion = 14f;
```

TanqueRuta.cs: dentro de OnTriggerEnter, debajo del if de la moneda

```
        if (other.CompareTag("Bomba"))
        {
            Explotar(other.gameObject);
        }
```

TanqueRuta.cs: función nueva

```
    void Explotar(GameObject bomba)
    {
        Vector3 centro = bomba.transform.position;
        Collider[] cercanos = Physics.OverlapSphere(centro, radioExplosion);

        for (int i = 0; i < cercanos.Length; i++)
        {
            Rigidbody otro = cercanos[i].attachedRigidbody;
            if (otro != null)
            {
                otro.AddExplosionForce(fuerzaExplosion, centro, radioExplosion, 0.3f, ForceMode.Impulse);
            }
        }

        Destroy(bomba);
    }
```

7 Quedar fuera: caerse o quedarse atrás

20 minutos · Comparar posiciones, bool, SetActive.

LO QUE DIGO

- El que se cae pierde. Y el que se queda atrás de la cámara también.
- ¿Cómo sabe el tanque que se cayó? Porque su altura baja. ¿Y que se quedó atrás? Comparando su posición con la del Foco.
- Cuando un tanque queda fuera, lo desactivamos y marcamos que ya no está vivo. Esa información la va a usar el marcador.

PREGUNTAS PARA EL GRUPO

- ¿A qué altura decido que ya se cayó?
- ¿Qué pasa si el tanque no sabe dónde está el Foco?
- ¿Por qué desactivar el tanque en vez de borrarlo?

LO QUE HACEN ELLOS

1. Agregar una variable para la distancia máxima que se puede quedar atrás.
2. En Update: si la altura del tanque es menor que -3, o si está más de esa distancia detrás del Foco, llamar a una función Eliminar.
3. En Eliminar: marcar vivo como falso y desactivar el tanque.
4. Arrastrar el objeto Foco al campo foco de los 4 tanques en el Inspector.

Pistas (sin dar el código)

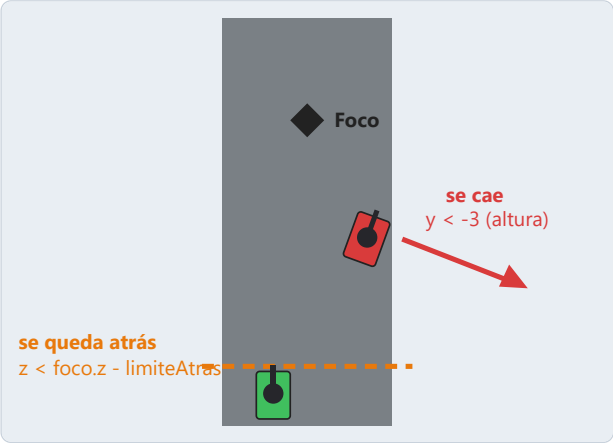
- transform.position.y es la altura. transform.position.z es la posición a lo largo de la carretera.
- foco.position.z es la posición del Foco.
- gameObject.SetActive(false) apaga el objeto.

Cómo compruebo que funciona

- Empujar a un tanque fuera de la carretera: cae y desaparece.
- Quedarse quieto: el tanque desaparece cuando el Foco se aleja 9 metros.

ERRORES TÍPICOS

- No arrastraron el Foco al campo foco: NullReference al darle Play.
- Usaron la Y del Foco en vez de la Z.



Hay dos formas de quedar fuera: caerse por el borde o quedarse atrás.

▼ Solución del paso 7 (mostrar al final)

TanqueRuta.cs: variable nueva

```
    public float limiteAtras = 9f;
```

TanqueRuta.cs: en Update, debajo de la lectura del giro

```
        if (transform.position.y < -3f)
        {
            Eliminar();
        }

        if (transform.position.z < foco.position.z - limiteAtras)
        {
            Eliminar();
        }
```

TanqueRuta.cs: función nueva

```
    void Eliminar()
    {
        vivo = false;
        gameObject.SetActive(false);
    }
```

8 Marcador y ganador

25 minutos · Referencias entre objetos, TextMeshPro, métodos con parámetros, contar.

LO QUE DIGO

- El marcador no controla a nadie: solo lee. Lee los puntos y si está vivo cada tanque, y lo escribe en pantalla.
- Como los 4 jugadores se muestran igual, escribo una función una sola vez y la uso 4 veces, cambiando el texto, el nombre y el tanque. Eso es un método con parámetros.
- Cuando solo quede uno vivo, el texto del centro anuncia al ganador.

PREGUNTAS PARA EL GRUPO

- ¿Qué tiene que saber el marcador para escribir un texto?
- ¿Cómo cuento cuántos siguen vivos?
- ¿Qué muestro si nadie queda vivo?

LO QUE HACEN ELLOS

1. Arrastrar al objeto Marcador: los 4 tanques, los 4 textos de las esquinas y el texto del centro.
2. En Update, escribir en cada texto el nombre del jugador y sus puntos. Si el tanque ya no está vivo, mostrar "fuera".
3. Contar cuántos tanques siguen vivos.
4. Si queda uno, escribir "Ganó" y el nombre en el texto del centro. Si no queda ninguno, "Nadie ganó".

Pistas (sin dar el código)

- Para escribir en un TextMeshPro: texto.text = "..." (hace falta using TMPro).
- Unir texto con números: "J1: " + puntos.
- Un método con parámetros: void MostrarJugador(TextMeshProUGUI texto, string nombre, TanqueRuta tanque).

Cómo compruebo que funciona

- Cada esquina muestra los puntos de su tanque.
- Cuando un tanque queda fuera, su texto cambia a "fuera".
- Cuando solo queda uno, aparece "¡Ganó J#!" en el centro.

ERRORES TÍPICOS

- Se les olvidó arrastrar algún texto o tanque: NullReference al darle Play.
- Arrastraron el texto de J2 en el campo de J1: los puntos aparecen en la esquina equivocada.

▼ **Solución del paso 8 (mostrar al final)**

MarcadorRuta.cs completo

```
using UnityEngine;
using TMPro;

public class MarcadorRuta : MonoBehaviour
{
    public TanqueRuta tanque1;
    public TanqueRuta tanque2;
    public TanqueRuta tanque3;
    public TanqueRuta tanque4;

    public TextMeshProUGUI texto1;
    public TextMeshProUGUI texto2;
    public TextMeshProUGUI texto3;
    public TextMeshProUGUI texto4;
    public TextMeshProUGUI textoCentral;

    void Update()
    {
        MostrarJugador(texto1, "J1", tanque1);
        MostrarJugador(texto2, "J2", tanque2);
        MostrarJugador(texto3, "J3", tanque3);
        MostrarJugador(texto4, "J4", tanque4);

        int vivos = 0;
        if (tanque1.vivo)
        {
            vivos++;
        }
        if (tanque2.vivo)
        {
            vivos++;
        }
        if (tanque3.vivo)
        {
            vivos++;
        }
        if (tanque4.vivo)
        {
            vivos++;
        }

        if (vivos == 0)
        {
            textoCentral.text = "Nadie ganó";
        }

        if (vivos == 1)
        {
            if (tanque1.vivo)
            {
                textoCentral.text = "¡Ganó J1!";
            }
            if (tanque2.vivo)
            {
                textoCentral.text = "¡Ganó J2!";
            }
            if (tanque3.vivo)
            {
                textoCentral.text = "¡Ganó J3!";
            }
            if (tanque4.vivo)
            {
                textoCentral.text = "¡Ganó J4!";
            }
        }
    }

    void MostrarJugador(TextMeshProUGUI texto, string nombre, TanqueRuta tanque)
    {
        if (tanque.vivo)
        {
            texto.text = nombre + ": " + tanque.puntos;
        }
        else
        {
            texto.text = nombre + ": fuera (" + tanque.puntos + ")";
        }
    }
}
```

9 Jugar, equilibrar y mejorar

30 minutos · Probar, ajustar valores, ideas propias.

LO QUE DIGO

- Ahora juguemos de verdad. Tres rondas.
- Después de cada ronda, cada grupo propone un ajuste: ¿qué estuvo muy fácil, muy difícil o injusto? Lo cambiamos en el Inspector y probamos otra vez.

PREGUNTAS PARA EL GRUPO

- ¿El mapa va muy rápido o muy lento?
- ¿Las bombas son muy fuertes?
- ¿Hay muchos o pocos objetos?

LO QUE HACEN ELLOS

1. Jugar 3 rondas.
2. Cambiar al menos 2 valores en el Inspector para equilibrar el juego y probar otra vez.
3. Elegir un reto extra de la lista y hacerlo.

Pistas (sin dar el código)

- Todos los valores están como variables públicas: no hace falta tocar el código para ajustarlos.

Cómo compruebo que funciona

- Hay una ronda completa con ganador.
- Cada grupo puede explicar qué cambió y por qué.

Retos extra

- Que al llegar a la cima gane quien tenga más puntos, aunque sigan varios vivos.
- Un objeto nuevo con otro tag, por ejemplo un turbo que dé velocidad extra por unos segundos.
- Que la bomba también cambie el color del tanque al que empuja.
- Agregar más tramos de carretera y más objetos.
- Cambiar la forma de mover el tanque (por ejemplo, con MovePosition) y comparar cómo se siente.

Los tres scripts: de Inicio a Final

Lo que traen los estudiantes y lo que tiene la solución. La última columna dice en qué paso se agrega cada parte.

Script	Inicio	Final	Pasos
TanqueRuta	foco, puntos y vivo (variables vacías)	Teclas, empuje, giro, monedas, bombas y quedar fuera	1, 2, 5, 6, 7
CamaraRuta	Nada (clase vacía)	Mover el Foco: espera, velocidad, aceleración y parada	4
MarcadorRuta	Referencias a los 4 tanques y a los 5 textos	Escribir los textos, contar vivos y anunciar al ganador	8

TanqueRuta

INICIO (LO QUE TRAEN)

```
using UnityEngine;

// META: convertir este tanque en un tanque jugable para varias personas en el mismo teclado.
// Cada tanque tiene sus propias teclas. Tiene que avanzar, retroceder y girar,
// empujar a los otros tanques al chocar y recoger monedas para ganar puntos.
// Si toca una bomba, la explosión empuja a los tanques que estén cerca.
// Si se cae de la carretera o se queda atrás de la cámara, queda fuera de la partida.
public class TanqueRuta : MonoBehaviour
{
    public Transform foco;
    public int puntos;
    public bool vivo = true;
}
```

▼ Final (la solución completa)

```
using UnityEngine;

public class TanqueRuta : MonoBehaviour
{
    public KeyCode teclaAdelante;
    public KeyCode teclaAtras;
    public KeyCode teclaIzquierda;
    public KeyCode teclaDerecha;

    public Transform foco;
    public float fuerza = 28f;
    public float velocidadGiro = 120f;
    public float limiteAtras = 9f;
    public float radioExplosion = 4f;
    public float fuerzaExplosion = 14f;
    public int puntos = 0;
    public bool vivo = true;

    private Rigidbody rb;
    private float empuje = 0f;
    private float giro = 0f;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        empuje = 0f;
        if (Input.GetKey(teclaAdelante))
        {
            empuje = 1f;
        }
        if (Input.GetKey(teclaAtras))
        {
            empuje = -1f;
        }

        giro = 0f;
        if (Input.GetKey(teclaDerecha))
        {
            giro = 1f;
        }
        if (Input.GetKey(teclaIzquierda))
        {
            giro = -1f;
        }

        if (transform.position.y < -3f)
        {
            Eliminar();
        }

        if (transform.position.z < foco.p
        {
            Eliminar();
        }
    }

    void FixedUpdate()
    {
        rb.AddForce(transform.forward * e

        Quaternion rotacion = Quaternion.
        rb.MoveRotation(rb.rotation * rot
    }

    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Moneda"))
        {
            puntos++;
            Destroy(other.gameObject);
        }

        if (other.CompareTag("Bomba"))
        {
            Explotar(other.gameObject);
        }
    }

    void Explotar(GameObject bomba)
    {
        Vector3 centro = bomba.transform.
        Collider[] cercanos = Physics.Ove
```

```
        for (int i = 0; i < cercanos.Leng
        {
            Rigidbody otro = cercanos[i].
            if (otro != null)
            {
                otro.AddExplosionForce(fu
            }
        }

        Destroy(bomba);
    }

    void Eliminar()
    {
        vivo = false;
        gameObject.SetActive(false);
    }
}
```

CamaraRuta

INICIO (LO QUE TRAEN)

```
using UnityEngine;

// META: que el mapa avance solo por la carretera, cada vez más rápido,
// y se detenga al llegar a la cima. Los tanques pueden ir más rápido que el mapa.
public class CamaraRuta : MonoBehaviour
{

}
```

▼ Final (la solución completa)

```
using UnityEngine;

public class CamaraRuta : MonoBehaviour
{
    public float velocidadInicial = 4f;
    public float aceleracion = 0.03f;
    public float velocidadMaxima = 9f;
    public float retardoInicio = 3f;
    public float zFinal = 380f;

    private float velocidad;
    private float tiempo = 0f;

    void Start()
    {
        velocidad = velocidadInicial;
    }

    void Update()
    {
        tiempo += Time.deltaTime;

        if (tiempo > retardoInicio && transform.posi
        {
            if (velocidad < velocidadMaxima)
            {
                velocidad += aceleracion * Time.delt
            }

            transform.position += Vector3.forward *
        }
    }
}
```

MarcadorRuta

INICIO (LO QUE TRAEN)

```
using UnityEngine;
using TMPro;

// META: que cada texto muestre los puntos de su tanque, o que ya quedó fuera.
// Cuando solo quede un tanque en la carretera, el texto del centro tiene que anunciar al ganador.
public class MarcadorRuta : MonoBehaviour
{
    public TanqueRuta tanque1;
    public TanqueRuta tanque2;
    public TanqueRuta tanque3;
    public TanqueRuta tanque4;

    public TextMeshProUGUI texto1;
    public TextMeshProUGUI texto2;
    public TextMeshProUGUI texto3;
    public TextMeshProUGUI texto4;
    public TextMeshProUGUI textoCentral;
}
```

▼ Final (la solución completa)

```
using UnityEngine;
using TMPro;

public class MarcadorRuta : MonoBeha
{
    public TanqueRuta tanque1;
    public TanqueRuta tanque2;
    public TanqueRuta tanque3;
    public TanqueRuta tanque4;

    public TextMeshProUGUI texto1;
    public TextMeshProUGUI texto2;
    public TextMeshProUGUI texto3;
    public TextMeshProUGUI texto4;
    public TextMeshProUGUI textoCent

    void Update()
    {
        MostrarJugador(texto1, "J1",
        MostrarJugador(texto2, "J2",
        MostrarJugador(texto3, "J3",
        MostrarJugador(texto4, "J4",

        int vivos = 0;
        if (tanque1.vivo)
        {
            vivos++;
        }
        if (tanque2.vivo)
        {
            vivos++;
        }
        if (tanque3.vivo)
        {
            vivos++;
        }
        if (tanque4.vivo)
        {
            vivos++;
        }

        if (vivos == 0)
        {
            textoCentral.text = "Nad
        }

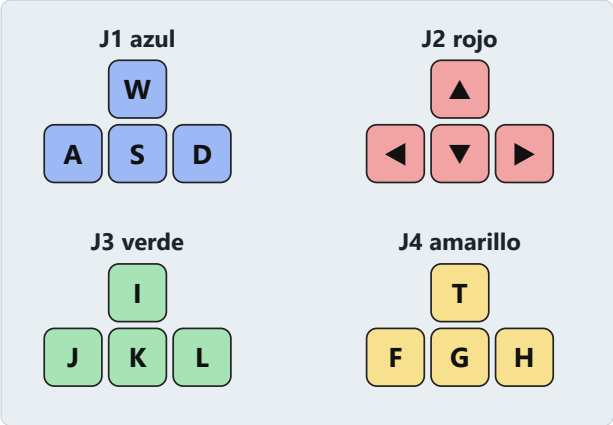
        if (vivos == 1)
        {
            if (tanque1.vivo)
            {
                textoCentral.text =
            }
            if (tanque2.vivo)
            {
                textoCentral.text =
            }
            if (tanque3.vivo)
            {
                textoCentral.text =
            }
            if (tanque4.vivo)
            {
                textoCentral.text =
            }
        }
    }

    void MostrarJugador(TextMeshProU
    {
        if (tanque.vivo)
        {
            texto.text = nombre + ":
        }
        else
        {
            texto.text = nombre + ":
        }
    }
}
```

Ajustes si no se siente bien

Problema	Variable	Dónde	Valor inicial
Los tanques son muy lentos o muy rápidos	fuerza	TanqueRuta (en cada tanque)	28
El mapa arranca o acelera muy rápido	velocidadInicial, aceleracion, velocidadMaxima	CamaraRuta (en el Foco)	4, 0.03, 9
Eliminan muy rápido a quien se atrasa	limiteAtras	TanqueRuta	9
Las bombas son muy fuertes	fuerzaExplosion, radioExplosion	TanqueRuta	14, 4
Cuesta girar	velocidadGiro	TanqueRuta	120
Se resbalan mucho o poco	Linear Damping	Rigidbody del tanque	1.5
Hay muchos o pocos objetos	mover o borrar	objetos en la escena	-

Las teclas



J1 azul, J2 rojo, J3 verde, J4 amarillo. Q y E no se usan en esta versión.

Nota

Los valores son una primera propuesta: hay que ajustarlos jugando. La escena final y los scripts finales los guardo solo yo; los estudiantes reciben el paquete de Inicio.